LangChain durch Literate Programming erlernen

1. Kontext und Idee

Literate Programming ist ein Konzept von Donald Knuth (1984): Code und Dokumentation sind nicht getrennt, sondern bilden dasselbe Dokument. Ein Jupyter Notebook ist die moderne Umsetzung davon – Markdown-Zellen erklären was passiert und warum, Code-Zellen zeigen es. Das Ergebnis ist lesbar wie ein Tutorial und ausführbar wie ein Programm.

LangChain ist ein Open-Source-Framework für Python (und JavaScript), das die Entwicklung von Applikationen mit grossen Sprachmodellen (LLMs) vereinfacht. Da verschiedene LLM-Anbieter unterschiedliche APIs und Antwortformate verwenden, standardisiert LangChain die Interaktion mit Sprachmodellen – ein Anbieterwechsel ist damit ohne grossen Aufwand möglich.

Ziel dieser Arbeit ist kein theoretisches Forschungsprojekt, sondern praktisches Hands-on-Lernen: Du baust vier aufeinander aufbauende Notebooks, die LangChain Schritt für Schritt erklären – und wendest dabei selbst die Prinzipien des Literate Programming an.

2. Zielsetzung

Am Ende der Arbeit liegen vier eigenständig lauffähige, vollständig dokumentierte Jupyter Notebooks vor, die zusammen einen roten Faden durch die zentralen LangChain-Konzepte bilden – von der ersten LLM-Anfrage bis zu einem kleinen, getesteten Assistenten. Die Notebooks sollen so geschrieben sein, dass eine aussenstehende Person mit Python-Grundkenntnissen LangChain allein anhand dieser Notebooks erlernen kann.

3. Der Use Case: der Modul-Assistent

Alle vier Notebooks bauen gemeinsam an einem einzigen, durchgehenden Beispiel, das Modul-Assistenten. Das ist ein Chat-Tool, das Fragen zu einem konkreten Modul beantwortet – zum Beispiel zu Inhalten aus Foliensätzen. Anders als ein reines Sprachmodell rät der Assistent nicht, sondern stützt seine Antworten auf echte Modulunterlagen.

Der Assistent wächst dabei Notebook für Notebook um jeweils eine Fähigkeit:

- **Notebook 01:** beantwortet eine erste Frage zum Modul mit strukturierter Antwort (Antwort + Quelle als festes Schema) – noch ohne Gedächtnis.
- **Notebook 02:** merkt sich den bisherigen Gesprächsverlauf innerhalb einer Sitzung.
- **Notebook 03:** stützt Antworten per RAG auf echte Dokumente statt auf Vermutungen.
- **Notebook 04:** erhält ein eigenes Tool und wird anhand einiger eigener Testfragen überprüft – fertiger Mini-Assistent.

Der Use Case bleibt bewusst klein: eine einzelne Funktion bzw. ein einfaches Skript, das RAG über 2 PDF-Dokumente (z. B. Foliensätze) und ein Tool kombiniert und lokal in Notebook 04 lauffähig ist.

Der Fokus liegt auf dem Verständnis von LangChain, nicht auf einer produktionsreifen Anwendung.

4. Technische Lernziele

Nach Abschluss der Arbeit kannst du die folgenden Konzepte nicht nur anwenden, sondern erklären und begründen, warum sie so aufgebaut sind:

Kompetenz	Konkrete Nachweisfähigkeit
LLM-Grundarchitektur	Den Weg einer Nachricht von Python-Code durch LangChain bis zum Modell und zurück lückenlos erklären – inklusive Token-Zählung und Kostenabschätzung.
Prompt Engineering & LCEL	PromptTemplates modular aufbauen, mit dem LCEL-Pipe-Operator (|) zu Chains verknüpfen und den Datenfluss im Debugger Schritt für Schritt verfolgen.
Konversationsgedächtnis	Die drei Memory-Typen (Buffer, Summary, Window) vergleichen und für einen gegebenen Usecase begründet auswählen.
RAG-Pipeline	Eine vollständige Retrieval-Augmented-Generation-Pipeline von Grund auf bauen: Dokument laden → chunken → embedden → VectorStore → Retrieval → LLM-Antwort mit Quellenangabe.
Agenten & Tools	Eigene Tools mit dem @tool-Decorator definieren, einen ReAct-Agenten damit ausstatten und den Thought → Action → Observation-Loop im Log lesen und interpretieren.
Evaluation & Observability	Eigene Testfragen definieren und die Outputs des Assistenten manuell (oder mit einem einfachen Skript) systematisch bewerten. LangSmith-Tracing als optionale Vertiefung, falls Zeit bleibt.
Architekturentscheidungen	Begründet entscheiden, wann eine statische Chain ausreicht und wann ein dynamischer Agent notwendig ist.

5. Methodische Anforderungen

Diese Anforderungen betreffen die Arbeitsweise und sind das Kernversprechen des Literate-Programming-Ansatzes: Code und Verständnis wachsen synchron.

- **Erklärung vor jeder Code-Zelle:** Die Markdown-Zelle direkt davor beantwortet „Was passiert hier?“ und „Warum so und nicht anders?“ – fehlende Erklärung bedeutet ein unvollständiges Notebook.
- **Konzept vor Code:** Jedes neue LangChain-Konzept wird zuerst in eigenen Worten beschrieben (mit Analogie oder Diagramm), bevor die erste Code-Zelle erscheint.
- **Iterationen sichtbar machen:** Der erste Prompt wird selbstständig verfasst, bevor eine KI zur Verbesserung hinzugezogen wird. Im Prompt-Logbuch werden nur der erste eigene Prompt und die finale Version festgehalten – der finale Prompt allein genügt nicht.
- **Fehler als Lehrinhalt:** Mindestens ein typischer Fehler pro Notebook wird mit absichtlich falschem Code reproduziert, die Fehlermeldung erklärt und die Korrektur begründet gezeigt.
- **Narrative Kohärenz:** Die vier Notebooks bilden einen roten Faden – jedes Notebook schliesst mit einer Vorschau auf das nächste, und Konzepte aus früheren Notebooks werden aktiv referenziert.
- **Reproduzierbarkeit:** Alle Notebooks laufen auf einer frischen venv ohne manuelle Eingriffe.

6. Aufbau jedes Notebooks

Jedes der vier Notebooks folgt demselben inneren Aufbau. Diese Struktur ist verbindlich und wird bei der Bewertung geprüft:

Notebook-Abschnitt	Inhalt & Anforderung
Lernziele	Markdown: 3–5 Bullet Points, was man nach diesem Notebook kann. Wird am Ende überprüft.
Setup-Zelle	Code: Imports, load_dotenv(), Versions-Check. Muss auf frischer venv funktionieren.
Konzept-Erklärung	Markdown: Das Kernkonzept erklärt, bevor auch nur eine Zeile Code erscheint. Analogien erwünscht.
Experiment-Block	Code + Markdown abwechselnd: Jede Code-Zelle hat eine Markdown-Zelle direkt davor, die erklärt was passiert und warum.
Häufige Fehler	Markdown + Code: Mind. 1 typischer Fehler mit absichtlich falschem Code, der den Fehler reproduziert, dann die Korrektur.
Prompt-Logbuch	Markdown: eigener erster Prompt (ohne KI-Hilfe) → finale, mit KI verbesserte Version → kurze Erkenntnis, was sich verändert hat und warum.
Zusammenfassung	Markdown: Was haben wir gebaut? Was kommt im nächsten Notebook?
Übungsaufgabe	Markdown + leere Code-Zelle: eine konkrete Aufgabe, die die Leserin selbst lösen soll.

7. Die vier Notebooks im Detail

Damit die Notebooks tatsächlich aufeinander abgestimmt sind (nicht nur der Form nach, sondern inhaltlich), verwenden alle vier dieselbe durchgehende Fallstudie: den Modul-Assistenten (siehe Abschnitt 3). Die letzte Spalte zeigt, welchen Teil des Assistenten das jeweilige Notebook baut – dadurch entsteht am Ende automatisch die geforderte Vorschau/Rückbezug-Struktur.

Notebook	Kernkonzepte	Beitrag zum Assistenten
01_hello_langchain_prompts.ipynb	**Erste Schritte, Prompts & LCEL** Warum LLMs zustandslos sind und wie das Message-Array (System/Human/AI) den Kontext trägt. Erste Aufrufe mit ChatOpenAI inkl. Streaming-Antworten und Token-Counting zur Kostenabschätzung. Danach: PromptTemplates variabel befüllen, mit dem LCEL-Pipe-Operator (|) zu einer Chain verbinden und die Antwort mit einem OutputParser in ein festes Format (Structured Output) bringen.	Assistent beantwortet eine erste Frage mit strukturierter Antwort (Antwort + Quelle als festes Schema).
02_memory.ipynb	**Konversationsgedächtnis** Warum ein LLM sich ohne Memory nicht an frühere Nachrichten erinnert. Vergleich der drei Memory-Typen anhand eines Entscheidungsbaums: Buffer (vollständiger Verlauf), Summary (komprimierte Zusammenfassung) und WindowBuffer (nur die letzten n Nachrichten) – inkl. session_id zur Trennung mehrerer Konversationen.	Assistent merkt sich den Gesprächsverlauf innerhalb einer Sitzung.

03_rag.ipynb	**RAG – Retrieval Augmented Generation** Wie ein Dokument in durchsuchbare Bausteine zerlegt wird: Laden (Loaders), Aufteilen (Splitter), in Vektoren umwandeln (Embeddings) und in einer VectorStore (ChromaDB) ablegen – mit Visualisierung, wie ähnliche Inhalte im Embedding-Raum nahe beieinander liegen. Anschliessend Retrieval der relevantesten Abschnitte zu einer Frage und Zusammenführung mit dem LLM über eine RetrievalQA-Chain.	Assistent stützt Antworten auf 1–2 echte Modulunterlagen im PDF-Format (z. B. Foliensatz und/oder Wegleitung) statt auf Vermutungen.
04_agents_evaluation.ipynb	**Agenten, Tools & Mini-Evaluation** Der ReAct-Loop (Thought → Action → Observation) im Detail: wie ein Agent entscheidet, ob und welches Tool er braucht, und wann eine statische Chain stattdessen ausreicht. Eigene Tools mit dem @tool-Decorator definieren, einem AgentExecutor übergeben und anhand selbst formulierter Testfragen prüfen, ob die Antworten des Assistenten sinnvoll sind.	Assistent erhält ein Tool und wird anhand eigener Testfragen überprüft – fertiger Mini-Assistent.

8. Abschlussreflexion

Nach Abschluss der vier Notebooks schreibst du eine kurze schriftliche Reflexion (ca. 1-2 Seiten) über deine Erfahrungen mit dem Projekt. Dabei gehst du auf folgende vier Punkte ein:

- **Technisches Verständnis:** Welches LangChain-Konzept hast du zu Beginn unterschätzt oder falsch verstanden – und wodurch hat sich das im Verlauf der Arbeit geändert?
- **Zusammenarbeit mit der KI:** Wie hat sich dein Vorgehen beim Prompten über die vier Notebooks hinweg verändert, seit du jeden ersten Prompt selbst schreibst und die KI erst danach zur Verbesserung hinzuziehst (vgl. Abschnitt 5, Prompt-Logbuch)?
- **Herausforderungen:** Welcher Teil war inhaltlich am schwierigsten zu verstehen oder umzusetzen, und was würdest du beim nächsten Mal anders angehen?
- **Der Modul-Assistent:** Wie hat sich deine Vorstellung vom fertigen Modul-Assistenten (Abschnitt 3) im Verlauf der Arbeit verändert?

9. Mögliche Erweiterungen

Beschreibe kurz zwei Erweiterungen, die du dem Modul-Assistenten oder den Notebooks hinzufügen würdest, wenn du mehr Zeit hättest – pro Vorschlag 2–3 Sätze: was die Erweiterung ist, und warum gerade diese (z. B. weil sie an eine Stelle anknüpft, die im Kernumfang bewusst offen gelassen wurde, oder weil du während der Arbeit ein konkretes Bedürfnis dafür gesehen hast).